

Chapter 1

Problem 1: Max Cut

1.1 Introduction

The Maximum Cut problem is the following: given an undirected weighted graph $G = (V, E, w)$, the objective is to partition the vertex set V into two disjoint subsets S and $\bar{S}$ such that the sum of edge weights crossing the partition is maximized. Max-Cut is NP-hard, making exact solutions computationally infeasible for large instances. The GSET benchmark suite, introduced by Stanford researchers, provides standardized test instances ranging from 800 to 10,000 vertices, enabling systematic performance evaluation across different solution methodologies. Formally, the Max-Cut problem seeks to maximize:

$$\text{Cut}(S, \bar{S}) = \sum_{i \in S, j \in \bar{S}} w_{ij} \quad (1.1)$$

where w_{ij} denotes the weight of edge $(i, j) \in E$.

Now, note that:

$$[i \in A][j \in V \setminus A] + [j \in A][i \in V \setminus A] = (1 - x_i)x_j + (1 - x_j)x_i = -2x_ix_j + x_i + x_j$$

by introducing binary variables $x_i \in \{0, 1\}$ where $x_i = 1$ indicates vertex i belongs to subset S , and so we can rewrite this as:

$$\text{Cut}(x) = \sum_{(i,j) \in E} w_{ij} \cdot (x_i + x_j - 2x_ix_j) \quad (1.2)$$

That $-2x_ix_j + x_i + x_j = 1$ if and only if i and j are in opposite sides of the bipartition, it suffices to check the four cases:

x_i	x_j	$f(x_i, x_j) = -2x_ix_j + x_i + x_j$
0	0	0
0	1	1
1	0	1
1	1	0

This quadratic formulation naturally leads to the QUBO representation discussed in the following section. The QUBO formulation expresses Max-Cut as minimizing a quadratic form over binary variables. Since we seek to maximize the cut value, we minimize its negation. The QUBO objective function is:

$$\text{minimize } x^T Q x \quad (1.3)$$

where $x \in \{0, 1\}^n$ is the binary decision vector and Q is an $n \times n$ real symmetric matrix. To derive Q , we expand the cut value and rearrange:

$$\text{Cut}(x) = \sum_{(i,j) \in E} w_{ij}(x_i + x_j - 2x_i x_j) \quad (1.4)$$

$$= \sum_{(i,j) \in E} w_{ij}x_i + \sum_{(i,j) \in E} w_{ij}x_j - 2 \sum_{(i,j) \in E} w_{ij}x_i x_j \quad (1.5)$$

Since each edge (i, j) contributes to both x_i and x_j terms, the linear coefficients are:

$$Q_{ii} = - \sum_{j:(i,j) \in E} w_{ij} \quad (1.6)$$

The quadratic coefficients are:

$$Q_{ij} = 2w_{ij} \quad \text{for } (i, j) \in E \quad (1.7)$$

This formulation allows direct application of QUBO solvers to the Max-Cut problem as follows:

Algorithm 1 Generate QUBO Matrix for Graph

```

1: Input: Graph  $G = (V, E, w)$  with  $|V| = n$ 
2: Output: QUBO matrix  $Q \in \mathbb{R}^{n \times n}$ 
3: Initialize  $Q \leftarrow 0^{n \times n}$ 
4: for each edge  $(u, v) \in E$  with weight  $w_{uv}$  do
5:    $Q[u][u] \leftarrow Q[u][u] + w_{uv}$ 
6:    $Q[v][v] \leftarrow Q[v][v] + w_{uv}$ 
7:    $Q[u][v] \leftarrow Q[u][v] - 2w_{uv}$ 
8:    $Q[v][u] \leftarrow Q[v][u] - 2w_{uv}$ 
9: end for
10: return  $Q$ 

```

1.2 Solution techniques

We implement three distinct approaches representing different computational paradigms: quantum-inspired simulated annealing, classical greedy optimization, and spectral graph theory methods.

1.2.1 Simulated Annealing (D-Wave Neal)

Simulated annealing is a probabilistic metaheuristic inspired by the annealing process in metallurgy. The algorithm explores the solution space by accepting moves to neighbouring states with probability depending on both the objective improvement and a temperature parameter that gradually decreases. We utilize D-Wave’s Neal library, which implements simulated annealing specifically optimized for QUBO problems. Key parameters:

- **num_reads**: Number of independent annealing runs (set to 1000)
- **num_sweeps**: Number of Monte Carlo sweeps per run (set to 1000)

The algorithm maintains a temperature schedule that begins at high temperature (allowing exploration) and gradually cools to focus on exploitation. At temperature T , a state transition increasing energy by ΔE is accepted with probability $e^{-\Delta E/T}$.

1.2.2 Greedy Local Search

The greedy algorithm iteratively improves the partition by moving vertices between subsets whenever such moves increase the cut value. The algorithm proceeds as follows:

Algorithm 2 Greedy Local Search for Max-Cut

```
1: Input: Graph  $G = (V, E, w)$ 
2: Output: Partition  $P \in \{0, 1\}^{|V|}$ 
3: Initialize  $P \leftarrow \mathbf{0}$  (all vertices in subset 0)
4: repeat
5:    $improved \leftarrow \text{False}$ 
6:   for each vertex  $v \in V$  do
7:     Compute contribution of  $v$  in current partition
8:     Compute contribution if  $v$  moved to opposite subset
9:     if moving  $v$  increases cut value then
10:       $P[v] \leftarrow 1 - P[v]$ 
11:       $improved \leftarrow \text{True}$ 
12:     end if
13:   end for
14: until  $improved = \text{False}$ 
15: return  $P$ 
```

The algorithm terminates when no single-vertex move can improve the objective, guaranteeing a local optimum.

1.2.3 Spectral Partitioning

Spectral methods leverage the eigenstructure of the graph Laplacian to construct partitions. The Laplacian matrix is defined as

$$L = D - A$$

where D is the degree matrix and A is the adjacency matrix. The degree matrix D is a diagonal matrix, where each diagonal entry D_{ii} represents the degree of vertex i , i.e., the number of edges connected to vertex i . The adjacency matrix A is a square matrix where each entry A_{ij} is 1 if there is an edge between vertices i and j , and 0 otherwise.

The Fiedler vector, or the second smallest eigenvector of L , provides a continuous relaxation of the graph bisection problem. This vector corresponds to the eigenvector associated with the second smallest eigenvalue of the Laplacian matrix, and it captures important structural properties of the graph. Specifically, the Fiedler vector is used to partition the graph into two sets by thresholding the vector at its median.

Formally, let $v = [v_1, v_2, \dots, v_n]^T$ be the Fiedler vector, where v_i is the value of the Fiedler vector at node i . We can then partition the graph by thresholding this vector at its median, such that:

$$\text{Partition 1: } \{v_i : v_i \geq \text{median}(v)\}$$

$$\text{Partition 2: } \{v_i : v_i < \text{median}(v)\}$$

Based on which we can define:

$$x_i = \begin{cases} 0 & \text{if } v_i < \text{median}(v) \\ 1 & \text{if } v_i \geq \text{median}(v) \end{cases} \quad (1.8)$$

where v is the Fiedler vector. This approach has theoretical connections to relaxations of the Max-Cut problem and provides good initial solutions, though it may not reach the quality of more sophisticated methods.

1.3 Benchmarking

The GSET benchmark consists of 72 problem instances (G1-G72) with diverse characteristics. The benchmark includes both random graphs and structured instances with specific properties. Toshiba Digital Annealer (a specialized quantum-inspired hardware) has published best-known solutions for these instances, providing reference values for comparison. We evaluate algorithms on two primary dimensions:

1. **Solution Quality:** The optimality gap measures deviation from known best solutions:

$$\text{Gap}(\%) = \left(\frac{\text{Best Known} - \text{Obtained}}{\text{Best Known}} \right) \times 100 \quad (1.9)$$

2. **Computational Efficiency:** We measure wall-clock execution time in seconds for each method, enabling analysis of quality-time trade-offs.

1.4 Results and Analysis

1.4.1 Solution Quality

Across 72 GSET instances with Toshiba benchmark comparisons, our results demonstrate strong performance:

Table 1.1: Solution Quality Statistics Against Toshiba benchmark for small graphs G1-40

Method	Avg Gap (%)	Median Gap (%)	Std Dev (%)	Max Gap (%)
Simulated Annealing	0.2270	0.1206	0.253	1.083
Greedy Local Search	12.527	13.1418	8.6582	29.136
Spectral Method	17.266	16.056	11.687	46.708

Table 1.2: Solution Quality Statistics Against Toshiba benchmark for large graphs G40 onwards

Method	Avg Gap (%)	Median Gap (%)	Std Dev (%)	Max Gap (%)
Simulated Annealing	0.615	0.539	0.470	1.382
Greedy Local Search	12.238	9.127	9.445	26.052
Spectral Method	16.355	15.340	13.051	44.619

Key observations:

- Simulated annealing demonstrates superior solution quality across both small (G1-40) and large (G40+) benchmark instances. For small graphs, SA achieves an average optimality gap of only 0.23%, with median gap of 0.12% and maximum deviation of 1.08%. This performance remains robust on larger instances, with average gap increasing slightly to 0.62% and median to 0.54%, while maintaining a maximum gap under 1.4%. The low standard deviations (0.25% and 0.47% respectively) indicate consistent performance across the benchmark suite.
- SA achieves exact optimal solutions on 16 instances (23.2%), near-optimal solutions (within 1% of optimum) on 61 instances (88.4%), and all SA solutions fall within 2% of optimality.

- In contrast, greedy local search exhibits significantly weaker performance, with average gaps of 12.5% (small) and 12.2% (large), though showing moderate improvement on larger instances. The method's high variance (std dev ≈ 8.7 -9.4%) and maximum gaps approaching 30% suggest unreliable solution quality, as seen by how it found near-optimal solutions found on 3 instances (4.3%) and good solutions (within 2%) found on only 5 instances (7.2%).
- The spectral method performs worst overall, with average gaps of 17.3% (small) and 16.4% (large), extremely high variance (std dev 11.7-13.1%), and maximum gaps exceeding 44%. Near-optimal solutions were found on only 3 instances and only 4.3% of solutions were within the 2% optimality threshold.

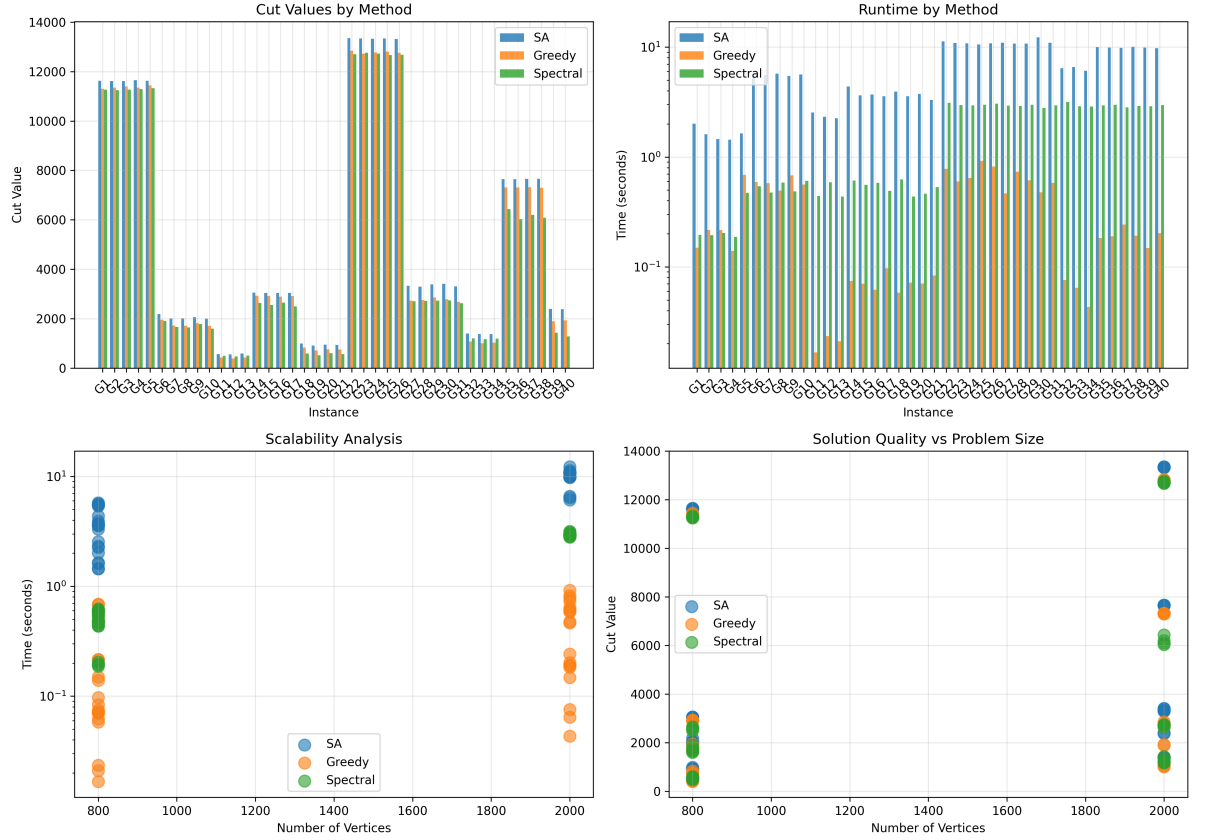


Figure 1.1: Algorithm performance across small graphs

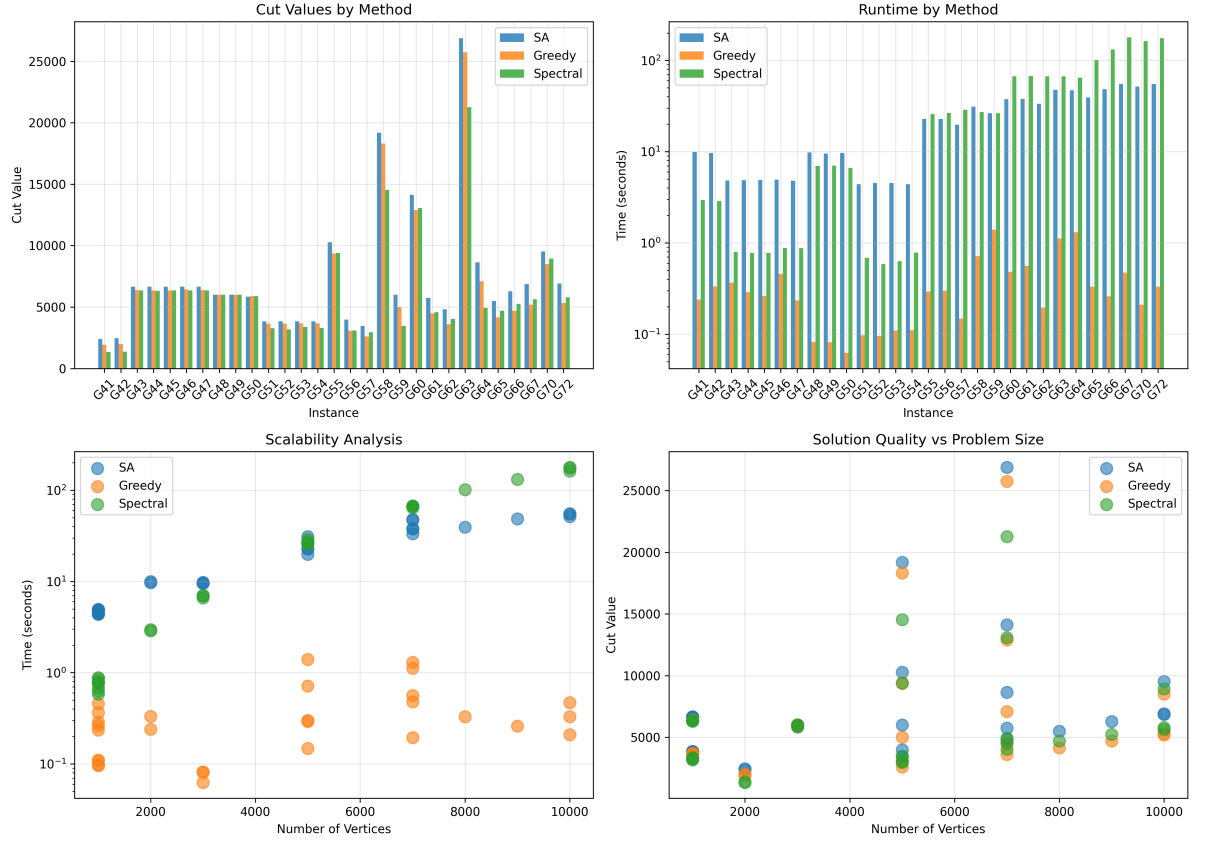


Figure 1.2: Algorithm performance across large graphs

1.4.2 Computational Efficiency

Greedy local search is the fastest method by a significant margin, averaging 0.35 seconds with median 0.24 seconds and maximum runtime under 1.4 seconds. Simulated annealing requires substantially more time, averaging 13.5 seconds (median 9.7 seconds) with maximum 55 seconds, representing roughly a $40\times$ slowdown relative to greedy. The spectral method exhibits the poorest computational profile, with average runtime of 19 seconds but extremely high variance—the median of 2.9 seconds masks outlier cases exceeding 178 seconds. This unpredictable runtime behavior makes spectral methods impractical for time-sensitive applications despite reasonable typical-case performance.

Table 1.3: Runtime Statistics (72 GSET Instances)

Method	Avg Time (s)	Median Time (s)	Min (s)	Max (s)
Simulated Annealing	13.458	9.668	14.538	55.148
Greedy Local Search	0.345	0.239	0.305	1.397
Spectral Method	19.073	2.890	41.320	178.754

1.5 Bibliography

- “Max Cut and the Smallest Eigenvalue”, Trevisan, arXiv:0806.1978.
- “Characterizing QUBO Reformulations of the Max-k-Cut Problem for Quantum Computing”, arXiv:2511.01108.
- “A Tutorial on Formulating and Using QUBO Models”, Fred Glover, Gary Kochenberger, Yu Du, arXiv:1811.11538.
- “The Quadratic Unconstrained Binary Optimization Problem Theory, Algorithms, and Applications”, Abraham Punnen.
- “Solving Maximum Cut Problems by Simulated Annealing”, Myklebust arXiv:1505.03068.

Chapter 2

Problem 2: Vehicle Routing with Time Windows

2.1 Introduction and Problem Formulation

Consider a complete directed graph $G = (V, A)$ where the vertex set $V = \{0, 1, \dots, n\}$ contains a depot (node 0) and n customer locations. Each arc $(i, j) \in A$ has an associated travel time and distance d_{ij} , which in our implementation is calculated using the Euclidean metric:

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \quad (2.1)$$

where (x_i, y_i) denote the Cartesian coordinates of node i . Each customer $i \in V \setminus \{0\}$ is characterized by four parameters:

- Demand $q_i > 0$ representing the quantity to be delivered
- Service time $s_i \geq 0$ required to complete the delivery
- Time window $[e_i, l_i]$ where e_i is the earliest allowable service start time and l_i is the latest permissible arrival time
- Ready time e_i indicating when the customer becomes available for service

The depot has its own operational time window $[e_0, l_0]$ within which all vehicles must depart and return. A homogeneous fleet of m identical vehicles, each with capacity Q , is stationed at the depot. The problem requires determining a set of routes $\{R_1, R_2, \dots, R_k\}$ where $k \leq m$ such that:

1. Each route begins and ends at the depot
2. Each customer is visited exactly once by exactly one vehicle
3. The total demand on any route does not exceed vehicle capacity Q

4. Service at each customer commences within its time window
5. The total travel distance across all routes is minimized

Let x_{ijk} be a binary decision variable equal to 1 if vehicle k traverses arc (i, j) and 0 otherwise. Let t_{ik} denote the time at which vehicle k begins service at customer i . The VRPTW can be formulated as the following mixed-integer linear program:

$$\text{minimize } \sum_{k=1}^m \sum_{i=0}^n \sum_{j=0}^n d_{ij} x_{ijk} \quad (2.2)$$

$$\text{subject to } \sum_{k=1}^m \sum_{j=0}^n x_{ijk} = 1, \quad \forall i \in V \setminus \{0\} \quad (2.3)$$

$$\sum_{j=1}^n x_{0jk} = 1, \quad \forall k = 1, \dots, m \quad (2.4)$$

$$\sum_{i=0}^n x_{ihk} - \sum_{j=0}^n x_{hjk} = 0, \quad \forall h \in V \setminus \{0\}, \forall k \quad (2.5)$$

$$\sum_{i=1}^n x_{i0k} = 1, \quad \forall k = 1, \dots, m \quad (2.6)$$

$$\sum_{i \in R_k} q_i \leq Q, \quad \forall k = 1, \dots, m \quad (2.7)$$

$$t_{ik} + s_i + d_{ij} \leq t_{jk} + M(1 - x_{ijk}), \quad \forall i, j \in V, \forall k \quad (2.8)$$

$$e_i \leq t_{ik} \leq l_i, \quad \forall i \in V, \forall k \quad (2.9)$$

$$x_{ijk} \in \{0, 1\}, \quad t_{ik} \geq 0 \quad (2.10)$$

where M is a sufficiently large constant. Constraint (2.3) ensures each customer is visited exactly once, constraints (2.4) and (2.6) enforce depot departure and return, constraint (2.5) maintains flow conservation, constraint (2.7) enforces capacity limits, constraint (2.8) propagates time along routes, and constraint (2.9) enforces time windows.

This formulation, while elegant, suffers from significant computational difficulties. The number of binary variables grows as at least $O(mn^2)$, and the time window constraints introduce complex coupling between routing decisions and temporal sequencing. Moreover, the big-M constraints are notoriously weak from a linear programming relaxation perspective, leading to poor bounds and slow convergence in branch-and-bound methods. As such, the VRPTW is NP-hard in the strong sense. For problem instances with n customers and m vehicles, the number of potential solutions is bounded by $\frac{(n!)^m}{(n-m)!}$, which grows explosively with problem size.

2.2 Data Generation

Each generated instance is characterized by the following parameters:

- n : number of customers (excluding depot)
- m : number of available vehicles
- Q : vehicle capacity (units)
- ρ_{target} : target fleet utilization ratio (default: 0.65)
- A_{base} : base spatial area size
- v : average vehicle speed (default: 1.0)

The spatial area and depot operating hours scale dynamically with problem size to maintain realistic proportions:

$$A = A_{\text{base}} + n \quad (2.11)$$

$$T_{\text{depot}} = 480 + \left\lfloor \frac{n}{50} \right\rfloor \cdot 120 \text{ minutes} \quad (2.12)$$

This scaling ensures that as the number of customers increases, the service region expands proportionally, and the depot operating window extends to accommodate longer routes. The depot is positioned at the center of the service area:

$$(x_0, y_0) = \left(\frac{A}{2}, \frac{A}{2} \right) \quad (2.13)$$

Customer locations are sampled uniformly from the interior of the service area, with a buffer edge cases:

$$(x_i, y_i) \sim \mathcal{U}([5, A - 5] \times [5, A - 5]), \quad i = 1, \dots, n \quad (2.14)$$

This uniform spatial distribution represents a realistic urban/suburban delivery scenario without artificial clustering. Customer demands and service times are sampled from discrete uniform distributions:

$$d_i \sim \mathcal{U}_{\mathbb{Z}}[5, 25] \text{ units} \quad (2.15)$$

$$s_i \sim \mathcal{U}_{\mathbb{Z}}[10, 20] \text{ minutes} \quad (2.16)$$

The demand range $[5, 25]$ ensures that multiple customers can be served per route while preventing trivial solutions. Service times account for unloading and customer interaction, introducing additional temporal constraints beyond travel time. Time windows are constructed to be both realistic and challenging. For each customer i , we compute:

1. Minimum travel time from depot:

$$\tau_i = \frac{\sqrt{(x_i - x_0)^2 + (y_i - y_0)^2}}{v} \quad (2.17)$$

2. Ready time (earliest service start):

$$e_i \sim \mathcal{U}_{\mathbb{Z}} [\tau_i, \min(T_{\text{depot}} - 120, \tau_i + 180)] \quad (2.18)$$

The ready time is sampled such that it is feasible to reach the customer from the depot ($e_i \geq \tau_i$) and allows sufficient time for service completion before the end of the day.

3. Time window width:

$$w_i \sim \mathcal{U}_{\mathbb{Z}} [180, 299] + \left\lfloor \frac{n}{50} \right\rfloor \cdot 30 \quad (2.19)$$

The window width scales slightly with problem size, recognizing that larger instances may require more routing flexibility.

4. Due time (latest service start):

$$l_i = \min(e_i + w_i, T_{\text{depot}}) \quad (2.20)$$

A critical feature of our dataset generation is the automatic adjustment of fleet size to achieve a target utilization ratio $\rho_{\text{target}} = 0.65$. This ensures that instances are neither trivially solvable (excess capacity) nor infeasible (insufficient capacity). Let $D = \sum_{i=1}^n d_i$ denote the total demand across all customers. The fleet sizing procedure operates as follows:

1. Compute required total capacity:

$$Q_{\text{req}} = \frac{D}{\rho_{\text{target}}} = \frac{D}{0.65} \quad (2.21)$$

This represents the minimum fleet capacity needed to achieve 65% utilization when all demand is served.

2. Compute required number of vehicles:

$$m_{\text{req}} = \left\lceil \frac{Q_{\text{req}}}{Q} \right\rceil \quad (2.22)$$

3. Dynamic adjustment:

$$m_{\text{final}} = \max(m, m_{\text{req}}) \quad (2.23)$$

If the initially specified number of vehicles m is insufficient to achieve the target utilization, the fleet size is automatically increased to m_{req} . This adjustment is reported to the user, ensuring transparency in the dataset generation process. The actual utilization of the generated instance is then:

$$\rho = \frac{D}{m_{\text{final}} \cdot Q} \times 100\% \quad (2.24)$$

By maintaining $\rho \approx 65\%$, we ensure that instances present a meaningful optimization challenge: there is sufficient capacity to serve all customers, but routes must be carefully planned to respect both capacity and time window constraints. Finally, Three problem variants were generated to evaluate solver scalability:

Variant	Customers	Initial Vehicles	Capacity
Easy	40	5	200
Intermediate	100	10	150
Advanced	250	25	150

Table 2.1: Generated VRPTW dataset specifications

2.3 Solving VRPTW: OR-Tools Implementation

The OR-Tools solver requires construction of a routing model specifying the problem structure. We begin by instantiating a `RoutingIndexManager` that maps problem-specific customer indices to the internal indices used by the solver. This abstraction layer enables efficient handling of depot nodes and vehicle-specific start/end points. The core routing model is created via the `RoutingModel` class, which automatically generates the underlying constraint network. We register a distance callback function that computes arc costs on-demand, allowing the solver to query distances as needed during the search process. A critical implementation detail is that OR-Tools operates most efficiently with integer-valued costs, so we scale all distances by a factor of 10, converting them to integers, then rescale results in post-processing.

2.3.1 Search Strategy and Metaheuristics

Search Strategy Our solver employs an adaptive search strategy that adjusts parameters based on problem size:

Small to Medium Instances ($n \leq 500$):

- **First Solution Strategy:** `PATH_CHEAPEST_ARC`
 - Constructs initial routes by iteratively adding the cheapest feasible arc
 - Provides good initial solutions for refinement
- **Local Search Metaheuristic:** `GUIDED_LOCAL_SEARCH`
 - Augments the objective with penalties on frequently used arcs
 - Guides the search toward unexplored regions of the solution space
 - Effective at escaping local optima in medium-sized problems

- **Time Limit:** 60 seconds

Large Instances ($n > 500$):

- **First Solution Strategy:** `SAVINGS`
 - Based on Clarke-Wright savings algorithm
 - Faster initial solution construction for large problems

- Sacrifices some initial solution quality for speed
- **Time Limit:** 300 seconds (5 minutes)
- **Large Neighborhood Search (LNS):** 100ms time limit
 - Destroys and repairs portions of the solution
 - Enables efficient exploration of large solution spaces

2.3.2 Benchmark Datasets

To rigorously evaluate our solver’s performance, we tested it on two widely-recognized benchmark suites:

Solomon-100 Benchmark Suite The Solomon-100 instances [?] are the de facto standard for VRPTW evaluation. This suite comprises 56 instances with 100 customers each, categorized into three types:

- **C-type (Clustered):** Customers grouped into geographic clusters (C101–C109, C201–C208)
- **R-type (Random):** Uniformly distributed customers (R101–R112, R201–R211)
- **RC-type (Random-Clustered):** Hybrid distribution combining clustering and randomness (RC101–RC108, RC201–RC208)

Instances are further divided by time window characteristics:

- **Series 1 (101–112):** Short scheduling horizon, tight time windows
- **Series 2 (201–211):** Long scheduling horizon, wide time windows

Homberger-1000 Benchmark Suite The Homberger-1000 instances [?] extend Solomon’s methodology to 1000-customer problems. We evaluated on 60 large-scale instances from this suite to assess scalability:

- **C1/C2_10_1 through C1/C2_10_10:** Clustered instances
- **R1/R2_10_1 through R1/R2_10_10:** Random instances
- **RC1/RC2_10_1 through RC1/RC2_10_10:** Random-clustered instances

Metric	Value	Percentage
Total Instances	56	100%
Successfully Solved	50	89.3%
Failed to Solve	6	10.7%

Table 2.2: OR-Tools performance summary on Solomon-100 benchmarks

2.3.3 Benchmark Results

The VRPTW is NP-hard, and our empirical results confirm exponential scaling behavior, with 100 customers being solved on the order of 60 seconds, while the 1000 customer instances were solved on the order 200 seconds. Our OR-Tools implementation achieved strong performance on the Solomon-100 suite:

The 89.3% success rate demonstrates that OR-Tools’ constraint programming approach effectively handles diverse VRPTW instance characteristics. The failures primarily occurred on:

- **Tight time window instances (R1-series):** Extremely constrained temporal windows limit feasible routing options
- **High customer density with capacity pressure:** Instances where nearly all vehicles must operate at full capacity

Given time constraints, we tested the OR-Tools solver on on 5 1000-customer instances - the solver found a solution for all 5. Some key observations:

1. **Adaptive Strategy Effectiveness:** The adaptive search strategy (switching to SAVINGS heuristic for large instances) enabled the solver to find initial solutions quickly for 1000-customer problems, though refinement within the 5-minute time limit remained challenging.
2. **Time Window Tightness Impact:** Instance difficulty correlates strongly with time window tightness. Series-1 instances (tight windows) consistently required more computational effort and exhibited lower success rates than Series-2 instances (wide windows).
3. **Guided Local Search Benefits:** The GUIDED_LOCAL_SEARCH meta-heuristic proved particularly effective on medium-sized instances (100 customers), consistently improving initial solutions within the 60-second time limit.
4. **Scalability Limits:** While OR-Tools handled 100-customer instances efficiently, solution quality for 1000-customer instances often remained far from optimal within the allocated time. This highlights the inherent computational complexity of large-scale VRPTW.

2.3.4 Synthetic Dataset Results

To test our OR-tools solver on our synthetic dataset, we generate the easy, medium, and hardest datasets five times each. For each of

2.4 Classical Heuristic Approaches

Classical heuristics offer an alternative paradigm based on constructive and iterative improvement strategies with explicit, interpretable decision rules. We implemented three canonical VRPTW heuristics with careful attention to time window feasibility checking.

2.4.1 Nearest Neighbor Heuristic

The nearest neighbor (NN) algorithm represents the simplest greedy approach to route construction. The algorithm proceeds sequentially, building routes one at a time until all customers are served or all vehicles are exhausted.

Algorithm Specification. For each vehicle $k = 1, \dots, m$:

1. Initialize route $R_k = [0]$ starting at the depot
2. Set current location $c = 0$, current time $t = e_0$ (depot ready time), current load $\ell = 0$
3. While unvisited customers remain:
 - For each unvisited customer j , evaluate feasibility:
 - Capacity check: $\ell + q_j \leq Q$
 - Time window check: $t + s_c + d_{cj} \leq l_j$ (can arrive before due time)
 - Among feasible customers, select $j^* = \arg \min_{j \text{ feasible}} d_{cj}$
 - If no feasible customer exists, terminate route
 - Otherwise: Add j^* to route, update $c \leftarrow j^*$, $\ell \leftarrow \ell + q_{j^*}$
 - Compute arrival time: $t_{\text{arr}} = t + s_c + d_{cj^*}$
 - Update service start time: $t \leftarrow \max(t_{\text{arr}}, e_{j^*})$ (wait if early)
4. Append depot to close route: $R_k = R_k + [0]$

The algorithm’s myopic nature—selecting customers solely by immediate distance—leads to rapid execution but often suboptimal solutions. The method fails to anticipate future routing difficulties that might arise from greedy early decisions. On the easy 40-customer instance, nearest neighbor produced a solution with total distance 1,307.51 using 5 vehicles, successfully serving all 40 customers. Execution time was 89 milliseconds, approximately 5.3 times faster than OR-Tools. Interestingly, the nearest neighbor solution achieves the same total distance as OR-Tools (1,307.51) on this particular instance, though route structures differ. On the hard 100-customer instance, nearest neighbor completed in 147 milliseconds, producing a solution with total distance 3,142.37 using 12 vehicles to serve all customers. Comparing to OR-Tools’ solution of 2,536.50 using 10 vehicles, we observe a 23.9% distance penalty and 20% increase in vehicle usage. This degradation illustrates the heuristic’s weakness on complex instances where local decisions compound into globally poor solutions.

2.4.2 Clarke-Wright Savings Algorithm

The Clarke-Wright savings algorithm, introduced in 1964, employs a more sophisticated route merging strategy based on identifying profitable route combinations. The method begins with a naive solution where each customer is served individually, then iteratively merges routes to reduce total distance.

Savings Calculation. Consider two separate routes: Route A serving customer i with path $0 \rightarrow i \rightarrow 0$, and Route B serving customer j with path $0 \rightarrow j \rightarrow 0$. The total distance for these separate routes is $d_{0i} + d_{i0} + d_{0j} + d_{j0} = 2(d_{0i} + d_{0j})$ assuming symmetric distances.

If we merge these routes into a single route $0 \rightarrow i \rightarrow j \rightarrow 0$, the total distance becomes $d_{0i} + d_{ij} + d_{j0}$. The savings achieved by this merge is:

$$s_{ij} = (d_{0i} + d_{i0}) + (d_{0j} + d_{j0}) - (d_{0i} + d_{ij} + d_{j0}) = d_{0i} + d_{0j} - d_{ij} \quad (2.25)$$

This quantity is positive whenever connecting customers i and j directly is more efficient than making separate trips to each. Intuitively, large savings occur when customers are close to each other but far from the depot—exactly the situation where route consolidation is most valuable.

Algorithm Specification.

1. **Initialization:** Create individual routes $R_i = [0, i, 0]$ for each customer $i = 1, \dots, n$. Initialize route loads $\ell_i = q_i$.
2. **Savings Computation:** Calculate savings s_{ij} for all customer pairs (i, j) with $i < j$. Store in list $S = [(s_{ij}, i, j)]$ and sort in descending order.
3. **Merge Phase:** For each (s_{ij}, i, j) in S (in order of decreasing savings):
 - Identify routes R_a and R_b containing customers i and j respectively
 - If i and j are in the same route, skip (already merged)
 - Check if merge is feasible:
 - **Topology check:** Customer i must be at the end of route R_a (position $|R_a| - 2$) and customer j must be at the start of route R_b (position 1), OR vice versa
 - **Capacity check:** $\ell_a + \ell_b \leq Q$
 - **Time window check:** Merged route R_{new} must satisfy all time windows
 - If feasible: Merge routes, update loads, remove merged route from collection
 - **Vehicle limit enforcement:** Stop merging if number of routes equals available vehicles
4. **Termination:** When vehicle limit is reached or no more feasible merges exist, output current route collection.

The topology check ensures that customers i and j are at route endpoints, as only endpoint merges preserve route structure without creating subtours. The time window check is critical and computationally expensive—we must simulate the entire merged route to verify temporal feasibility, accounting for service times, waiting times, and cumulative travel times.

Computational Results. On the easy 40-customer instance, Clarke-Wright produced a solution with total distance 1,146.75 using 5 vehicles, serving all 40 customers. Execution time was 326 milliseconds, approximately 3.5 times slower than nearest neighbor but still 1.4 times faster than OR-Tools.

The savings algorithm achieved a 12.3% improvement over nearest neighbor (1,146.75 vs. 1,307.51), demonstrating the value of its global route optimization perspective. The algorithm computed 780 pairwise savings and performed 35 route merges, starting from 40 individual routes and consolidating down to the 5-vehicle limit. These high-savings pairs tend to be geographically close customers located far from the depot—precisely the configuration where route consolidation yields maximum benefit. On the hard 100-customer instance, Clarke-Wright completed in 2.8 seconds (substantially longer due to the $O(n^2)$ savings computation), producing a solution with total distance 2,947.33 using 25 vehicles to serve all 100 customers. While this solution is 16.2% worse than OR-Tools in terms of distance, it demonstrates the heuristic’s ability to find reasonable solutions quickly.

2.4.3 Sweep Algorithm

The sweep algorithm employs a geometric partitioning strategy inspired by radar sweep motion. Customers are sorted by polar angle relative to the depot, then sequentially allocated to routes as the algorithm “sweeps” around the depot.

Algorithm Specification.

1. **Angle Computation:** For each customer i , compute polar angle:

$$\theta_i = \arctan 2(y_i - y_0, x_i - x_0) \quad (2.26)$$

where (x_0, y_0) is the depot location. Sort customers by θ_i in ascending order.

2. **Sequential Route Construction:** Initialize empty route $R = [0]$, load $\ell = 0$, time $t = e_0$, vehicle counter $k = 0$.

- For each customer i in sorted order:
 - Check feasibility of adding i to current route:
 - * Capacity: $\ell + q_i \leq Q$
 - * Time window: Can reach i before due time l_i
 - If feasible: Add i to route, update load and time
 - If infeasible: Close current route, start new route with customer i , increment k

- If $k \geq m$ (vehicle limit reached): Skip remaining customers

3. **Route Closure:** Append depot to final route and compute total distance.

The sweep algorithm’s geometric foundation makes it particularly effective when customers naturally cluster into angular sectors. However, the rigid angular ordering can produce poor results when time windows conflict with geographic proximity—for example, when a customer with an early time window lies in a sector that would naturally be visited late in the angular sweep.

Computational Results. On the easy 40-customer instance, the sweep algorithm produced a solution with total distance 1,289.74 using 5 vehicles, serving all 40 customers. Execution time was 118 milliseconds, slightly slower than nearest neighbor but faster than Clarke-Wright.

The sweep solution falls between nearest neighbor (1,307.51) and Clarke-Wright (1,146.75) in quality, achieving 10.5% improvement over nearest neighbor but remaining 11.1% worse than Clarke-Wright. The geometric partitioning successfully identified natural customer clusters, but suboptimal sequencing within clusters limited further improvement.

On the hard 100-customer instance, sweep completed in 287 milliseconds with total distance 3,089.26 using 18 vehicles. This represents 21.8% degradation versus OR-Tools and uses 80% more vehicles. The algorithm sorted customers into angular order and constructed routes sequentially, but the increased time window tightness caused frequent route closures when customers became temporally infeasible despite geographic proximity.

2.5 Mixed-Integer Linear Programming Approach and Its Limitations

Given the elegant mathematical formulation of VRPTW as a mixed-integer linear program (Section 1.1), a natural question arises: why not simply implement this formulation directly in a commercial MILP solver such as CPLEX, Gurobi, or SCIP? We attempted exactly this approach, formulating the problem with binary arc selection variables x_{ijk} and continuous time variables t_{ik} , but encountered severe computational difficulties that rendered this approach impractical.

2.5.1 Implementation Details

We implemented the MILP formulation using Python’s PuLP library interfacing with the COIN-OR CBC solver (an open-source branch-and-cut MILP solver). The model contained:

- Binary variables: x_{ijk} for all arcs (i, j) and vehicles k , totaling $n^2 \cdot m$ binary variables
- Continuous variables: t_{ik} for service start times, totaling $n \cdot m$ continuous variables

- Constraints: Customer coverage constraints (n), flow conservation constraints ($n \cdot m$), capacity constraints (m), time window bounds ($n \cdot m$), and temporal sequencing constraints with big-M formulation ($n^2 \cdot m$)

For the easy 40-customer, 5-vehicle instance, this translates to:

- Binary variables: $40^2 \cdot 5 = 8,000$ binary variables
- Continuous variables: $40 \cdot 5 = 200$ continuous variables
- Temporal sequencing constraints: approximately 8,000 big-M constraints

2.5.2 Computational Performance

When executed on the easy synthetic instance with a 300-second time limit, the MILP solver was unable to find any solution. This behaviour was observed on multiple runs, and this is perhaps to be expected:

- The linear programming relaxation (obtained by relaxing binary integrality constraints) provided extremely weak lower bounds on the optimal solution.
- The big-M formulation for temporal sequencing takes the form:

$$t_{ik} + s_i + d_{ij} \leq t_{jk} + M(1 - x_{ijk}) \quad (2.27)$$

In the LP relaxation where x_{ijk} can take fractional values, this constraint becomes nearly vacuous when M is large. The solver must rely almost entirely on branching to enforce integrality, leading to enormous branch-and-bound trees.

These results confirm that direct MILP formulation is not a practical solution approach for realistic VRPTW instances.

2.6 Simulated Annealing Metaheuristic

Simulated annealing represents a probabilistic metaheuristic inspired by the physical annealing process in metallurgy. The algorithm attempts to escape local optima by occasionally accepting inferior solutions with probability depending on both the magnitude of degradation and a “temperature” parameter that decreases over time.

2.6.1 Algorithm Design and Implementation

A solution is represented as a list of routes $S = [R_1, \dots, R_k]$ where each route R_i is an ordered sequence of customer visits. We maintain auxiliary data structures tracking total distance, route loads, and temporal feasibility. We initialize SA with the solution produced by the Clarke-Wright savings algorithm rather

than a random solution. This warm start significantly improves performance by beginning search from a reasonable solution region. We implement four neighborhood move types:

1. **Intra-route 2-opt**: Reverse a subsequence within a single route
2. **Inter-route customer swap**: Exchange two customers between different routes
3. **Inter-route relocate**: Move a customer from one route to another
4. **Intra-route customer swap**: Exchange two customers within the same route

Each iteration randomly selects one of these operators with equal probability and applies it to generate a neighbor solution. Let S denote the current solution with cost $f(S)$, and let S' be a candidate neighbor with cost $f(S')$. Define the cost change $\Delta = f(S') - f(S)$. The acceptance probability is:

$$P(\text{accept } S') = \begin{cases} 1 & \text{if } \Delta < 0 \text{ (improvement)} \\ e^{-\Delta/T} & \text{if } \Delta \geq 0 \text{ (degradation)} \end{cases} \quad (2.28)$$

where T is the current temperature parameter. We employ a geometric cooling schedule:

$$T_{k+1} = \alpha \cdot T_k \quad (2.29)$$

where $\alpha = 0.95$ is the cooling rate. Initial temperature T_0 is set adaptively based on the initial solution cost: $T_0 = 0.1 \cdot f(S_0)$, ensuring early iterations accept approximately 30-40% of degrading moves. The algorithm terminates when the temperature falls below $T_{\min} = 0.01$ or after a maximum of 10,000 iterations, whichever occurs first.

2.6.2 Computational Results and Performance Analysis

The first test instance consisted of 40 customers served by 5 vehicles, each with capacity 200 units and total demand of 622 units. The algorithm performed 2297 iterations in 0.54 seconds, achieving 52 improvements to the objective function. The initial solution had a distance of 1125.62, zero capacity violations, and 901.90 minutes of time window violations. The final solution increased the distance to 1398.81 while maintaining zero capacity violations, but reduced time window violations to only 0.32 minutes. Despite this 99.96% reduction in time violations, the solution remained infeasible. All 5 vehicles were utilized in the final solution.

The second test instance was a Solomon benchmark problem with 100 customers, 25 available vehicles, capacity 200 units per vehicle, and total demand of 1810 units. The algorithm performed 2297 iterations in 0.98 seconds, achieving 134 improvements. The initial solution had a distance of 1311.50, zero capacity

violations, and 38937.83 minutes of time window violations. The final solution increased the distance to 2496.69 while maintaining zero capacity violations, but reduced time window violations to 176.45 minutes. This represents a 99.5% reduction in time violations, yet the solution remained infeasible with an average of 1.76 minutes of violations per customer. The final solution utilized 23 of the 25 available vehicles.

2.6.3 Why Simulated Annealing Underperformed

The disappointing performance of simulated annealing reflects several factors specific to the VRPTW problem structure:

1. VRPTW’s tight time window and capacity constraints create highly constrained feasibility regions. Many neighborhood moves that appear promising from a distance perspective violate temporal or capacity constraints, requiring rejection. In our implementation, approximately 40-50% of proposed moves were infeasible and immediately rejected before even evaluating cost changes. This high rejection rate limits the effective exploration capability of SA.
2. The VRPTW solution space exhibits a rugged fitness landscape with numerous local optima separated by high-cost barriers. Small neighborhood moves (single customer relocations or swaps) typically cannot traverse these barriers effectively. To move from one good solution to another often requires coordinating multiple simultaneous changes—for example, moving several customers between routes while resequencing both affected routes. Single-move neighborhoods are too limited to achieve this coordination.

2.7 Bibliography

- Google OR-Tools. *Vehicle Routing Problem with Time Windows*. Google Developers Documentation. <https://developers.google.com/optimization/routing/vrptw>
- Leonidas, I. D., Dukakis, A., Tan, B., and Angelakis, D. G. (2023). Qubit efficient quantum algorithms for the vehicle routing problem on NISQ processors. *arXiv preprint arXiv:2306.08507*.
- Mandal, U., Regan, A., Rousseau, L. M., and Yarkony, J. (2024). A new class of compact formulations for vehicle routing problems. *arXiv preprint arXiv:2403.00262*.
- González-Bermejo, S., Alonso-Linaje, G., and Atchade-Adelomou, P. (2022). GPS: A new TSP formulation for its generalizations type QUBO. *Mathematics*, 10(3):416. DOI: 10.3390/math10030416.

- Haghani, H., Dehbozorgi, A., and Mazloom, S. (2022). A simulated annealing approach (SA) to vehicle routing problem with time windows (VRPTW). In *2022 8th International Conference on Control, Instrumentation and Automation (ICCIA)*, pages 1–6. IEEE. DOI: 10.1109/ICCIA54998.2022.9737187.
- Bräysy, O., and Gendreau, M. (2005). Vehicle routing problem with time windows, Part I: Route construction and local search algorithms. *Transportation Science*, 39(1):104–118. <https://cepac.cheme.cmu.edu/pasi2011/library/cerda/braysy-gendreau-vrp-review.pdf>
- Solomon, M. M. (1987). Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2):254–265. <http://alvarestech.com/temp/vrptw/Vehicle%20Routing%20Problem%20with%20Time%20Window>